



Week 4 Lab – Generating C4 Diagrams

In this lab, we will use an AI assistant to **generate** C1 (context) and C2 (container) models for three unfamiliar systems – and, more importantly, to **verify** them. Producing a diagram now takes seconds. Deciding whether it is the *right* diagram is the part that still belongs to you.

We will work in the **Structurizr DSL**, the reference implementation of the C4 model, so that a single model produces both the C1 and the C2 view. This is important, as it is the same idea at two levels of abstraction, not two unrelated drawings.

Before You Start

Two browser tabs are all you need: there is nothing to install and no account to create.

1. Open the Structurizr DSL playground: playground.structurizr.com (the shorter address structurizr.com/dsl also takes you there).
2. Open any AI chat assistant, e.g. the SMU Gemini subscription, Claude.ai, or whichever you prefer. You will not need a coding assistant for this lab.
3. In the playground, the DSL goes in the editor on the left. Press the ► (Render) button to see your diagrams and use the drop-down at the top to switch between the views your model defines.
4. Nothing is saved automatically. Keep your DSL in a text file as you go – you will want it again in Part 4.

A note on why we ask the AI for a DSL rather than a picture. A text DSL is diffable in a pull request, editable line by line, and regenerated when the design changes. It also forces the assistant to commit to a *model* – a set of elements and relationships it must name explicitly – instead of an ambiguous drawing where you cannot tell whether a box is a system, a container, or a wish.

Part 1 – Warm-Up: StudySpace (guided)

Read the following customer brief.

StudySpace

The university wants a system that lets students find and book study rooms across campus. Students search for a free room by building, date and time, book it, and check in on arrival by scanning a QR code on the door. If nobody checks in within fifteen minutes, the booking is released so that someone else can use the room.

Students book from a browser or from their phone. Facilities officers maintain the list of rooms and review no-show reports. Everyone signs in with their existing university account, and the doors are opened by the campus door-access system that is already installed. Reminders and confirmations go out by email and SMS.

Now work through the following steps.

5. Give your AI assistant the brief together with a prompt along these lines (feel free to improve on it):

You are helping me model a software system using the C4 model. Below is a customer brief. Produce a Structurizr DSL workspace that defines a systemContext view and a container view.

Rules:

- The system we are building is a single softwareSystem.
 - Anything we do not build, own or operate is a separate softwareSystem, tagged "External System".
 - Every container must have a technology.
 - Every relationship must be labelled with its purpose.
- Output the DSL only.

Brief: <paste the brief here>

6. Paste the DSL into the playground and press Render. If it does not parse, paste the error message back to the assistant and ask it to fix the DSL – this alone is a useful signal about how confidently AI produces syntax it has not checked.
7. Look at the two views (via the dropdown box) before you read any further. Write down anything you think is wrong with the AI-generated model.
8. Now compare against the reference model below. Yours will differ: that's fine and expected. What matters is whether the differences are defensible.

```
workspace "StudySpace" "Campus study room booking system" {

    model {
        student = person "Student" "Finds, books and checks in to study rooms."
        officer = person "Facilities Officer" "Manages room inventory and reviews no-shows."

        studyspace = softwareSystem "StudySpace" "Lets students find, book and check in to campus study rooms." {
            spa = container "Single-Page Application" "Provides booking and room management in the browser." "JavaScript / React"
            mobile = container "Mobile App" "Provides booking and QR check-in on the move." "Kotlin / Swift"
            api = container "API Application" "Enforces booking rules, availability and check-in." "Python / Flask"
            worker = container "Reminder Worker" "Sends reminders and releases no-show bookings." "Python / APScheduler"
            db = container "Database" "Stores rooms, bookings and check-in records." "PostgreSQL" {
```

```

        tags "Database"
    }
}

sso = softwareSystem "University Single Sign-On" "Authenticates staff and
students." {
    tags "External System"
}
doors = softwareSystem "Door Access System" "Unlocks study room doors for valid
bookings." {
    tags "External System"
}
notify = softwareSystem "Notification Service" "Sends email and SMS messages." {
    tags "External System"
}

student -> spa "Books and cancels rooms using"
student -> mobile "Books and checks in using"
officer -> spa "Manages rooms and reviews no-shows using"

spa -> api "Makes API calls to" "JSON / HTTPS"
mobile -> api "Makes API calls to" "JSON / HTTPS"
api -> db "Reads from and writes to" "SQL / TCP"
worker -> db "Reads from and writes to" "SQL / TCP"
api -> sso "Validates sign-in tokens with" "OIDC / HTTPS"
api -> doors "Requests door unlocks from" "JSON / HTTPS"
worker -> notify "Sends reminders and confirmations using" "REST / HTTPS"
}

views {
    systemContext studyspace "SystemContext" "The environment StudySpace operates
in." {
        include *
        autolayout tb
    }

    container studyspace "Containers" "The containers that make up StudySpace." {
        include *
        autolayout tb
    }

    styles {
        element "Person" {
            shape person
            background #08427b
            color #ffffff
        }
        element "Software System" {
            background #1168bd
            color #ffffff
        }
        element "External System" {
            background #999999
            color #ffffff
        }
        element "Container" {
            background #438dd5
            color #ffffff
        }
        element "Database" {
            shape cylinder
        }
    }
}

```

```

    }
}

```

Notice what you did *not* have to write. The relationships are declared once, between people and containers. Structurizr then infers the system-level relationships for the C1 view. One model, two levels of abstraction.

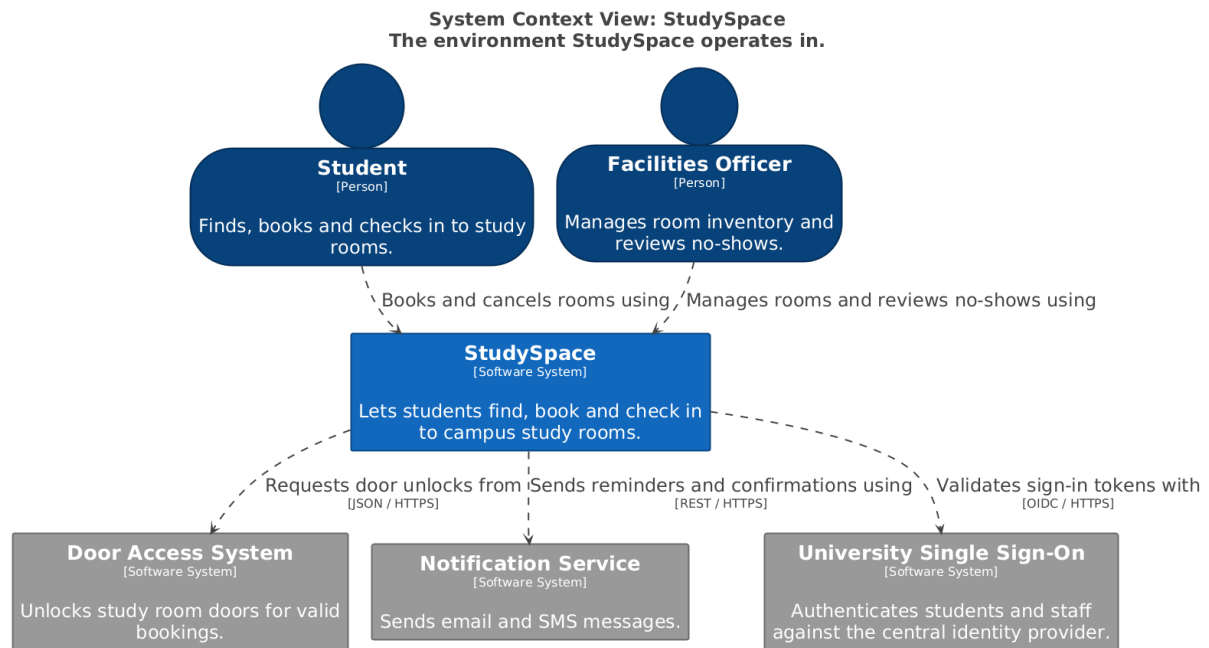


Figure 1 – C1 (System Context) view, rendered from the model above.

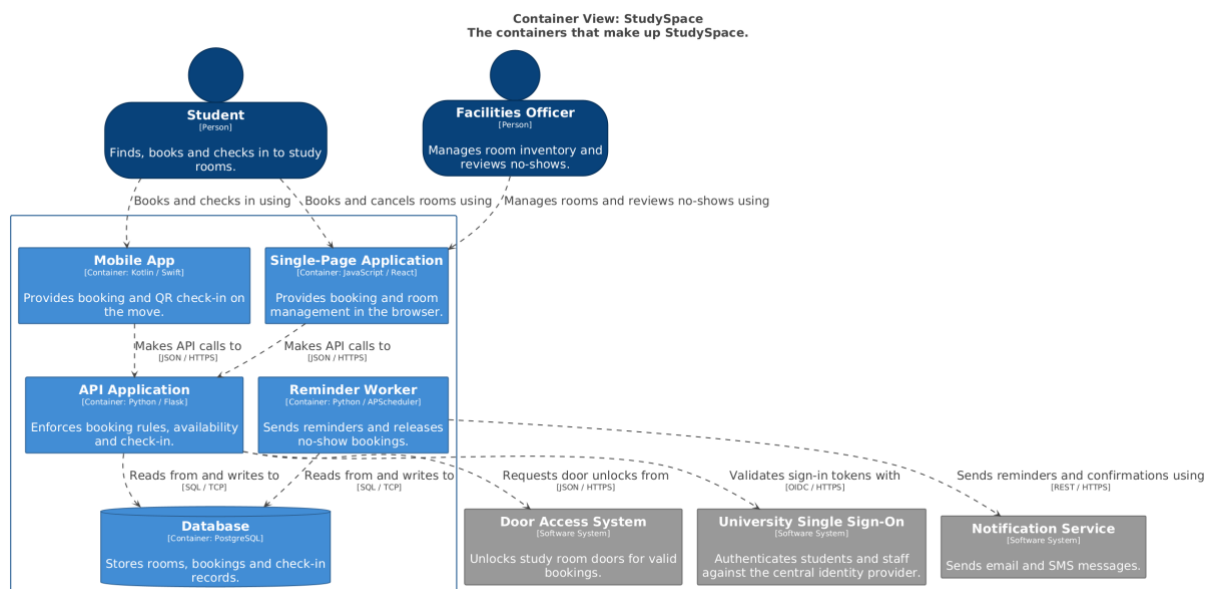


Figure 2 – C2 (Container) view, rendered from the same model.

Four things AI assistants routinely get wrong on a brief like this one:

- **Internals leak into C1.** The database, the API and the mobile app appear in the context diagram. C1 shows your system as *one box*.

- **Technology appears in C1.** Protocols and stacks belong at C2 and below. C1 has to be readable by someone who does not write code.
- **External systems get pulled inside the boundary.** Single sign-on and the door-access system are not yours; they are systems you depend on. More on this in Part 2.
- **The Reminder Worker goes missing.** No user clicks it, so it is easy to overlook – but a scheduled process that runs on its own is a container, and it is the thing that implements the fifteen-minute release rule.

One more, worth stating plainly: a container in the C4 model is an application or a data store. It is *not* necessarily a Docker container, and "one container per box" is not a deployment decision you have made yet.

Your Verification Checklist

Use this on every model you generate for the rest of the lab, and on your project in Part 4. The playground will catch structural mistakes for you: it will not let a person be a container, or an external system sit inside your boundary. What it cannot tell you is whether the boundary is in the right place, or whether something is missing. That part is yours. ☺

C1 – Context

Check	What a failing diagram looks like
Your system is a single box, with no internal structure shown.	Databases, services or apps that belong to your system appear as separate boxes.
Every other box is either a person (a role, not a named individual) or a system you do not own.	A box that your team would build, deploy and be paged about at 3am sits outside your system.
No technology, protocols or stacks appear anywhere on the diagram.	"PostgreSQL", "REST", "React" or "AWS" on a C1.
Every relationship reads as an intent, in the direction of the dependency.	Unlabelled arrows, or labels like "connects to" and "interacts with".
A non-technical stakeholder could read it without you narrating.	You need a paragraph of explanation before it makes sense.

C2 – Containers

Check	What a failing diagram looks like
Every container is a separately running process or a data store.	A class, module, controller or library has been promoted to a container – that is a C3 component.
Every container carries a technology label.	Generic boxes labelled "Backend" or "Service" with no technology.

The people and external systems from C1 are all still present, still outside the boundary.	The context quietly changed between the two views.
Relationships state both purpose and protocol or format.	"Uses" with no indication of how, or of what flows.
Every container exists because this brief needs it.	A cache, queue or API gateway that appears because it is a common pattern, not because anything in the brief asked for it.

Questions to ask of any AI-generated model

Question	Why it matters
Did it invent anything the brief never mentioned?	Plausible additions are the hardest kind of error to spot, because they look like expertise.
Did it automatically choose a technology stack on your behalf?	A default stack presented as a decision your team has already made.
What is missing that no description could have told it?	Batch jobs, regulatory feeds, legacy systems, out-of-hours processes – see Part 3.
Can you defend every box to the customer?	If you cannot say why a box is there, it is not your model yet.

Part 2 – TransitPal: Where Does Your System Stop?

TransitPal

A start-up wants to build a mobile app that helps Singapore commuters decide when to leave. Commuters enter where they are going and see live bus arrival times, how crowded each service is, and a warning if rain is forecast on their walking leg. They can save favourite journeys and receive a push notification when it is time to set off.

Live bus arrivals and crowding levels come from LTA's DataMall service. Walking directions and map tiles come from a commercial mapping provider. Forecasts come from the national weather service's public API. Push notifications are delivered through Apple's and Google's notification services. DataMall is rate-limited, so the team intend to poll it on a schedule and serve commuters from their own copy of the data. The team also want a small internal dashboard showing which routes are searched most often.

9. Generate a Structurizr DSL model for TransitPal and render both views.
10. Run the full checklist over it.

11. Then apply the “**paging test**” to every box inside your system boundary: if this thing broke at 3am, would someone on the TransitPal team be woken up to fix it? If the answer is no, it does not belong inside the boundary.
12. Fix the model, re-render, and note which boxes moved.

Watch for two traps in particular. The first is that some of the systems TransitPal depends on are large and famous, and assistants tend to draw famous things as though they were part of whatever is being described. The second is the reverse error: the mobile app *is* a container – the team writes it, ships it and fixes it – even though it runs on a phone they do not own.

Part 3 – GreenMeter: What the Brief Does Not Say

GreenMeter

The university’s facilities team wants a system to monitor energy use across campus buildings. Sensors in each building report electricity consumption every fifteen minutes. Facilities officers view dashboards showing consumption by building, floor and time of day, and set thresholds so that an alert is raised when a building exceeds its expected usage. The sustainability office wants to export monthly summaries as spreadsheets.

13. Generate and render C1 and C2 for GreenMeter, and run the checklist. Do not read the next box until you have done this.
14. Your model, and your assistant’s, will look perfectly reasonable. Now read what the customer mentioned three weeks into the project.

Three weeks later, the customer adds...

1. The sensor readings are often late or missing, so every night a reconciliation job compares them against the utility provider’s meter-reading feed and corrects the stored figures. Nobody mentioned it because "that is just how the data works".
2. The monthly summaries are not only internal. A formatted quarterly submission goes to a government agency, and the agency fixes the schema.
3. There is already a Building Management System that owns the sensors. GreenMeter never talks to a sensor directly – it reads from the BMS.
4. Alerts raised outside office hours do not go to facilities officers. They go to the security team’s existing incident system.

15. Update your model and re-render. Note which of the four changed C1, which changed C2, and which changed both.
16. For each of the four, write down the question you could have asked that would have surfaced it.

17. Then answer the question from lecture, in a sentence or two of your own: for C1 and C2, why is access to the repository – or to a written description – rarely enough?

This is the point of the whole lab. The first GreenMeter model was not badly drawn; it was confidently drawn, internally consistent, and wrong in ways no amount of prompting would have fixed. Architecture at C1 and C2 encodes *who owns what, what already exists, and what the organisation is obliged to do*, and none of that is in the code or the brief. It is in people's heads, and you get at it by asking.

Part 4 – Your Own Project System

The project instructions are out, so finish with your own system. This is worth **doing as a team** rather than individually.

18. Draft a C1 for your project system. You should be able to agree on this now: who uses it, and what it depends on that you are not building.
19. Draft a C2 with what you currently know. You will not have settled every technology choice yet – mark the uncertain containers with a tag "Undecided" or a note in the description rather than inventing a stack.
20. Commit the .dsl file to your repository, alongside the code, i.e. a text file that you regenerate when a decision changes.
21. Re-run the checklist on your own model. Be harder on yourselves than you were on the AI.

We are deliberately stopping at C2. C3 is only worth drawing when a particular container is complex enough that its internals need explaining: if that happens in your project, that is the moment to add one.

One last thought to carry into the project. Your C1 and C2 are the map an agent navigates by and are worth committing to your repo. An assistant working inside a legible structure asks better questions and makes smaller mistakes than one turned loose in a big ball of mud. If an AI agent joined your team tomorrow, could it find its way around your system? 😊

There is nothing to submit for this lab. If you get stuck on the DSL syntax, the playground links to the Structurizr DSL cookbook, and #help-is212 on Slack is open as always. 